

Rapport Saé 2.03

Paul BOONE, Lucas DUBOIS

2026-03-27

Contents

1	Préparation de la machine virtuelle	4
1.1	Création de la machine virtuelle	4
1.1.1	Dans VirtualBox	4
1.1.2	Questions	5
1.2	Préparation de l'OS	6
1.2.1	Récupérer l'OS	6
1.2.2	Configurer l'OS sur la machine virtuelle	6
1.2.3	Configuration des utilisateurs	8
1.2.3.1	SuperUtilisateur (root)	8
1.2.3.2	Autres utilisateurs	8
1.2.4	Partitionnement	8
1.2.5	Miroir	9
1.2.6	Logiciels	9
1.2.7	Questions	9
2	Préparation du système	11
2.1	Accès SUDO pour user	11
2.1.1	Se connecter en ROOT	11
2.1.2	Donner les droits SUDO au compte user	11
2.1.3	Se reconnecter et recevoir les droits	11
2.1.4	Questions	11
2.2	Installation des suppléments invités	12
2.2.1	Questions	12
3	Distribution Debian	13
3.1	Quelques questions sur Debian	13
3.1.1	La maintenance	13
3.1.2	Nom générique, nom de code et version	14
4	Automatisation	15
4.1	Récupérer et préparer les fichiers nécessaires	15
4.1.1	Création de la VM	15
4.1.2	Configurations	15
4.1.3	Questions	16
5	Analyse préliminaire de Git et des outils graphiques associés	17
5.1	Configuration globale de Git	17
5.1.1	Questions	17
5.2	Protocoles réseau pour Git	18
5.2.1	Questions	18
5.3	Manipulation pratique de Git	20
5.3.1	Questions	20
5.4	Travail collaboratif	22
5.4.1	Questions	22
6	Les interfaces graphiques pour Git	22
6.1	Questions	22
7	Installons autre chose et comparons	23
7.1	Questions	23
8	Préliminaire : redirection de ports	26
8.1	Questions	27

9	Installation de Forgejo	28
9.1	Installation de Forgejo	28
9.1.1	Installation des dépendances	28
9.1.2	Télécharger le binaire	28
9.1.3	Créer l'utilisateur système Git	28
9.1.4	Créer les répertoires	28
9.1.5	SQLite	29
9.1.6	Installer le service systemd	29
9.1.7	Configuration initiale via le web	29
9.1.8	Verrouiller la configuration	31
9.2	Architecture et protocoles réseaux	31
9.2.1	Questions	31
9.3	Base de donnée	31
9.3.1	Questions	31
9.4	Gestion du service Systemd	32
9.4.1	Questions	32
9.5	Sécurité	33
9.5.1	Questions	33
9.6	Tests d'utilisation	34
9.6.1	Création d'un dépôt depuis l'interface web	34
9.6.2	Ajout des rapports	34
9.6.3	Les codes sources des TPs	34
9.6.4	Problèmes rencontrés pendant les tests	36
9.6.4.1	Obligation de modifier le mot de passe avant une quelconque action	36
9.6.5	Création d'une issue	36
9.6.6	Ajout d'une clé SSH	37
9.6.7	Questions	37
10	Pour aller plus loin	38
10.1	Workflow Forgejo Actions	38
10.1.1	Création du workflow	38
10.1.2	Création du runner	39
10.2	Questions	40

1 Préparation de la machine virtuelle

1.1 Création de la machine virtuelle

1.1.1 Dans VirtualBox

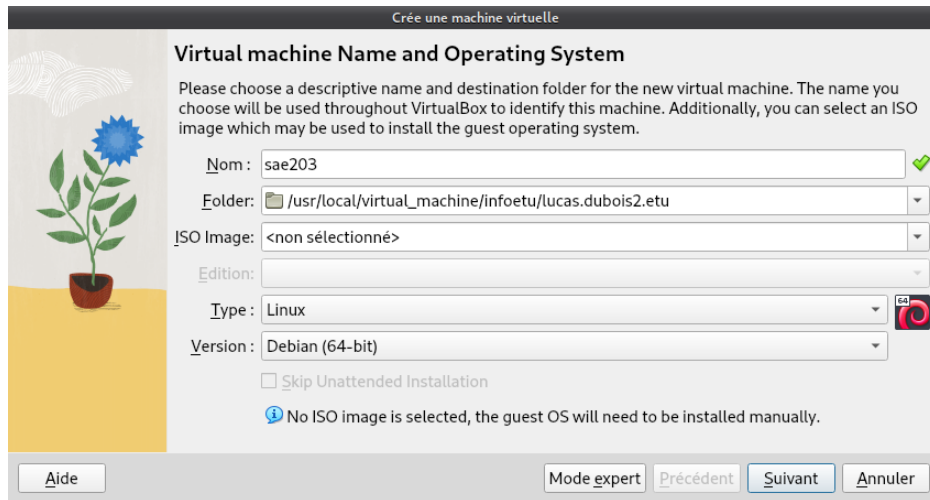


Figure 1: Préparation de la VM

entrer les informations de la machine :

- nom
- folder
- type
- version

puis appuyer sur suivant

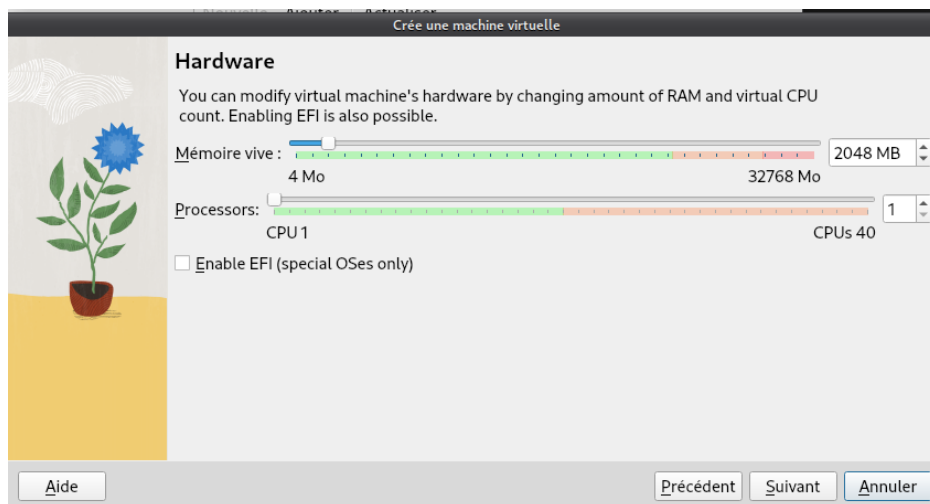


Figure 2: Préparation du Hardware de la VM

ne rien changer et faire suivant

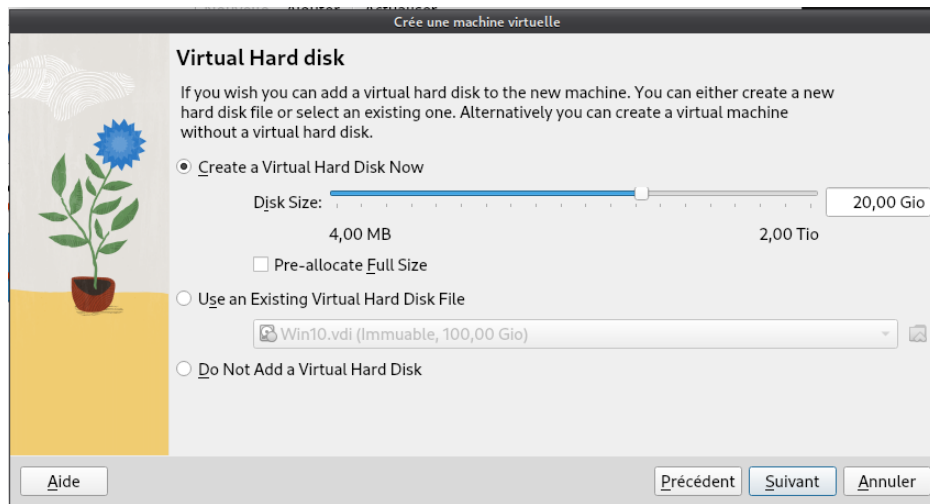


Figure 3: Configuration du Virtual Hard Disk de la VM

Ne toucher à rien, vérifier que la case

Pre-allocate Full Size

est bien décochée, cliquer sur suivant.

Ensuite cliquer sur finish.

1.1.2 Questions

Q1 : Que signifie “64-bit” dans “Debian 64-bit” ?

Le 64 bit dans Debian 64 bit indique que cette version de l’OS peut accéder à un total de 2^{64} adresses mémoires, soit 16 milliards de gigaoctets de mémoire. D’un point de vue matériel, les systèmes 32 bit peuvent avoir un maximum de 4 Go de RAM, alors que les systèmes 64 bit n’ont pas réellement de limite¹

Source : Microsoft, différences entre 32 bits et 64 bits

Q2 : Quelle est la configuration réseau utilisée par défaut ?

Le mode réseau par défaut est le NAT. VirtualBox crée un réseau virtuel privé pour la VM. Du coup, sur le réseau local de la machine physique, la VM n’a pas sa propre adresse IP visible, seulement une adresse dans le réseau virtuel de VirtualBox.²

Source : IT Connect, comprendre les différents types de réseaux VirtualBox

Q3 : Pourquoi allouer 2048 Mo de RAM à la VM ? Qu’est-ce qui se passerait avec seulement 512 Mo ?

On a mis 2048 Mo parce que c’est ce que recommande Debian pour faire tourner MATE correctement avec des applications comme Firefox ou LibreOffice.

Avec 512 Mo ça serait vraiment lent. Le système utiliserait le disque dur comme mémoire vive (swap) ce qui ralentit énormément. Pour un serveur sans interface graphique ça peut passer, mais pas pour notre utilisation.

Source : Site Debian, page Installation Guide, System Requirements

Q4 : Quel est le mode réseau utilisé par défaut par VirtualBox (NAT, pont, réseau interne...) ?

¹Il est impossible d’avoir 16 milliards de giga octet de RAM à l’heure actuelle

²Cette méthode peut être contournée en ajoutant une règle de redirection de port dans la configuration de la machine virtuelle

Le mode réseau par défaut est le NAT. VirtualBox crée un réseau virtuel privé pour la VM. Du coup, sur le réseau local de la machine physique, la VM n'a pas sa propre adresse IP visible, seulement une adresse dans le réseau virtuel de VirtualBox.³.

Source : IT Connect, comprendre les différents types de réseaux VirtualBox

Q5 : Quelle est l'adresse IP de votre VM ? Comment l'avez-vous obtenue ?

10.0.2.15 car c'est l'adresse IP par défaut attribuée à une VM avec le NAT.

Pour vérifier on peut taper `ip a` dans le terminal de la vm.

Source : VirtualBox Manual

Source : Oracle Documentation

Q6 - Quelle est l'adresse de la passerelle par défaut ? A quoi correspond cette adresse ?

La passerelle par défaut est 10.0.2.2⁴. On peut la retrouver avec la commande : `ip route`. C'est le routeur NAT virtuel de VirtualBox. Tout le trafic qui sort de la VM passe par lui. Il fait ensuite la traduction d'adresses (NAT) en utilisant l'adresse IP réelle de la machine hôte pour communiquer avec le monde extérieur.

1.2 Préparation de l'OS

1.2.1 Récupérer l'OS

Aller sur

Debian.org

et cliquer sur télécharger



Figure 4: Site de Debian

1.2.2 Configurer l'OS sur la machine virtuelle

Lancer la machine virtuelle en appuyant sur démarrer

³Cette méthode peut être contournée en ajoutant une règle de redirection de port dans la configuration de la machine virtuelle

⁴Fait intéressant : 10.0.2.2 donne aussi accès à l'interface loopback de l'hôte.

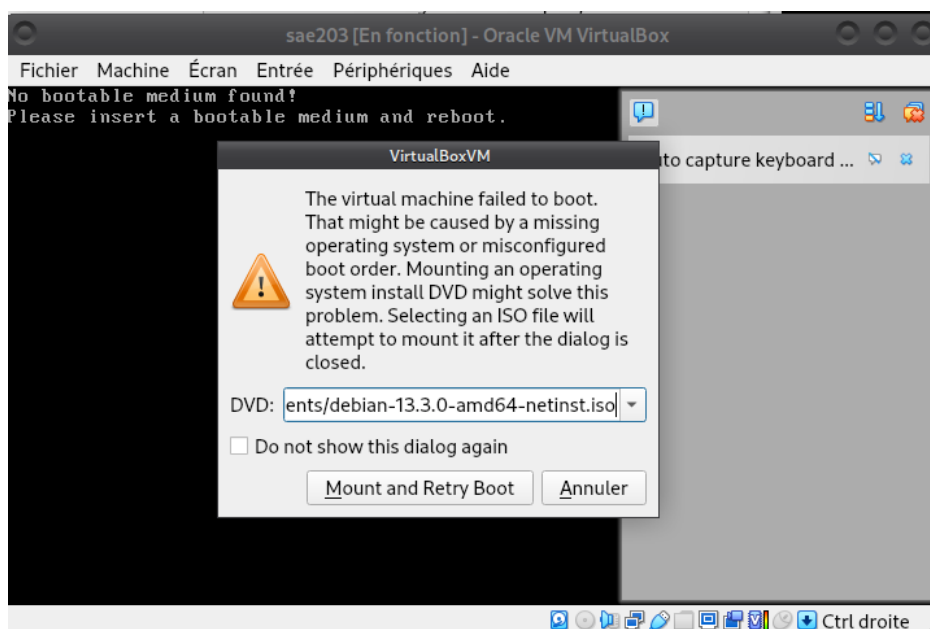


Figure 5: Installation de l'ISO

cliquer sur la petite flèche et sélectionner l'ISO que vous venez de télécharger

Cliquer sur “Mount and Retry Boot”

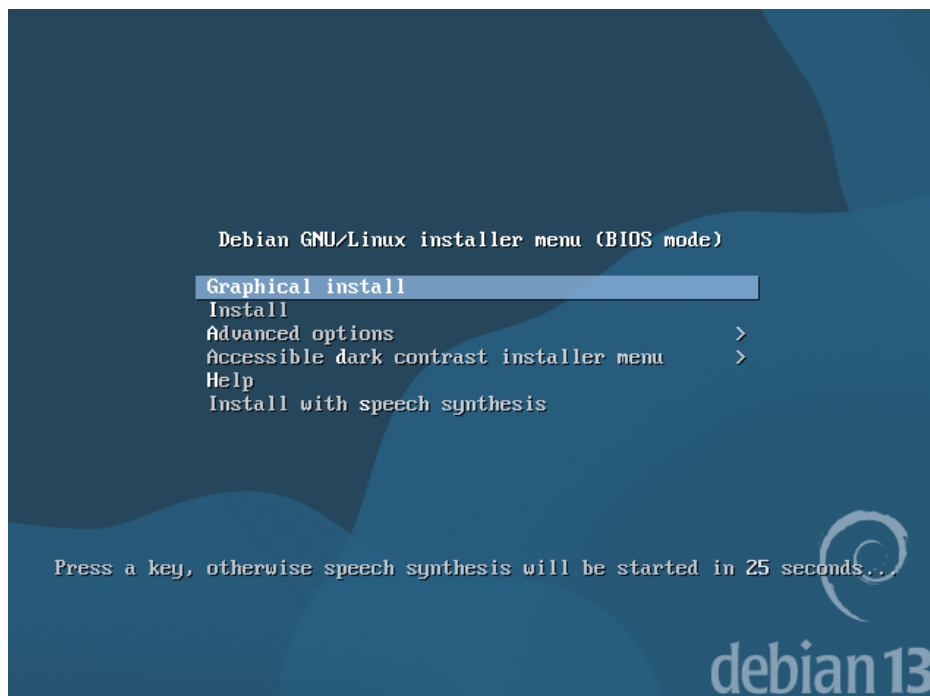


Figure 6: Sélectionner Graphical Install

Configurer ensuite la machine en langue française.

1.2.3 Configuration des utilisateurs

1.2.3.1 SuperUtilisateur (root)

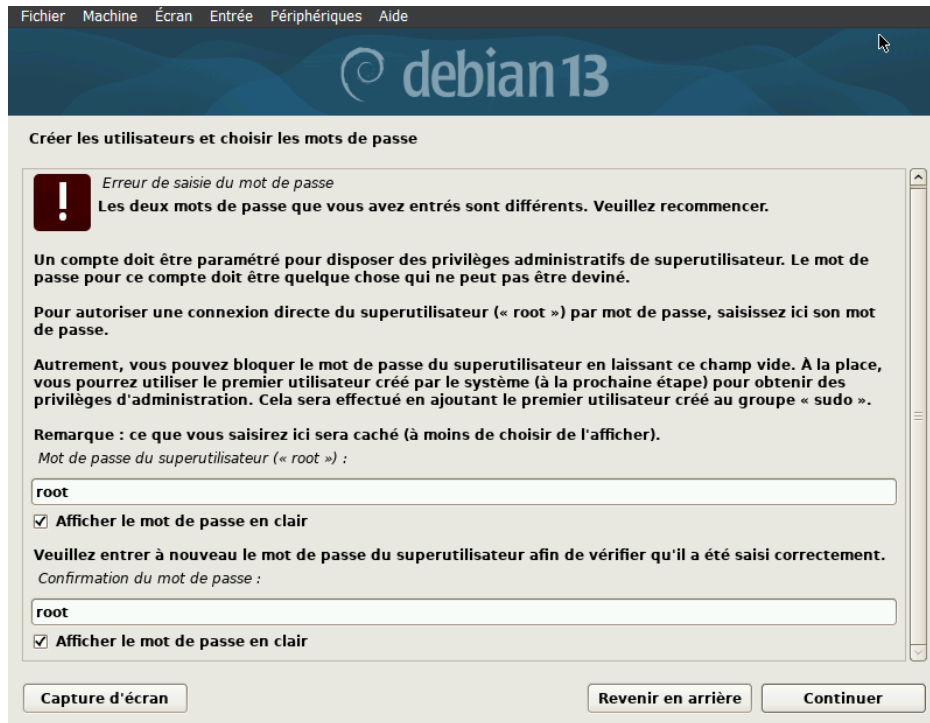


Figure 7: Configuration du superUser

Taper “root” dans les deux champs puis valider.

1.2.3.2 Autres utilisateurs

Pour le deuxième utilisateur entrer simplement les informations demandées ensuite (“user” partout ici).

1.2.4 Partitionnement

Pour le partitionnement de la machine sélectionner simplement **Utiliser un disque entier** puis **Tout dans une seule partition**. Cliquer sur terminer puis oui.

1.2.5 Miroir



Figure 8: Configuration des miroirs

sélectionner `debian.polytech-lille.fr` et ne choisir aucun mandataire.

1.2.6 Logiciels

```
78
79 ## Installation meta-paquetages
80 # Tâches à installer (via des méta-paquetages)
81 # lister les possibilités : tasksel --list-task (en ligne de commande)
82 # Utiliser au minimum "standard" est une bonne idée
83 tasksel tasksel/first multiselect standard ssh-server mate-desktop
84 d-i pkgssel/include string git sudo sqlite3 curl bash-completion neofetch
```

Figure 9: Configuration des logiciels

Cocher les logiciels suivants :

- environnement de bureau Debian
- MATE (penser à décocher Gnome)
- serveur web
- serveur ssh
- utilitaire usuels du système

Pour l'installation de GRUB, sélectionner le disque proposé par le système.

1.2.7 Questions

Question 1 : Qu'est ce qu'un fichier ISO bootable ?

Un fichier **ISO**, du format ISO 9660 ou 13346, contient toutes les données que l'on souhaite transférer sans les compresser (contrairement à un fichier d'archive). Il contient fréquemment toutes les données et la structure nécessaires pour lancer un système d'exploitation ou un programme depuis un support externe (CD, DVD, Clé USB).

Source : Ionos, Qu'est ce qu'un fichier ISO bootable

Question 2 : Qu'est-ce que MATE ? GNOME ?

MATE et GNOME sont des environnements de bureau pour Linux. Ce sont les interfaces graphiques qui permettent d'utiliser des fenêtres, des menus, etc. GNOME est plus moderne mais plus gourmand en ressources, MATE est plus léger (c'est un fork de l'ancien GNOME 2)..

Source : Wikipedia, GNOME

Question 3 : Qu'est-ce qu'un serveur mandataire ?

Un proxy (ou serveur mandataire) sert d'intermédiaire entre le client et internet. Il reçoit les requêtes du client, les transmet au serveur web, puis renvoie la réponse.

Il y a deux types :

1. Proxy direct : gère les requêtes du client vers internet
2. Proxy inverse : reçoit les requêtes d'internet et les envoie vers un serveur interne

Source : Mozilla Developer Network

Question 4 : Qu'est-ce qu'un serveur web ?

Un serveur web c'est à la fois :

Côté matériel : la machine physique qui héberge les fichiers du site

Côté logiciel : le programme (comme Apache ou Nginx) qui gère les requêtes HTTP et envoie les pages web aux visiteurs

Il existe deux types :

Type	Description
Statique	Envoie les fichiers tels quels (HTML, images, CSS...)
Dynamique	Génère les pages à la volée avec des bases de données et du code (PHP, Python...)

Source : Mozilla Developer Network

Source : Mozilla Developer Network |

Question 5 : Quel logiciel de serveur SSH a été installé ? Comment vérifier qu'il est bien démarré ?

Le logiciel installé est OpenSSH Server (paquet openssh-server sur Debian). C'est l'implémentation libre et la plus répandue du protocole SSH.

Pour vérifier qu'il est bien démarré, plusieurs commandes sont possible :

- `systemctl status ssh`

- `ps aux | grep sshd`

Source : Man sshd(8)

Source : OpenSSH

Question 6 : Testez une connexion SSH depuis votre machine hôte vers votre VM. Quelle commande utilisez-vous ? Quel problème rencontrez-vous et pourquoi ?

On essaierait normalement :

```
ssh user@10.0.2.15
```

Mais ça ne marche pas car en NAT l'adresse 10.0.2.15 n'existe que dans le réseau virtuel de VirtualBox. L'hôte ne peut pas la voir directement.

Pour que ça marche il faut faire une redirection de port dans les paramètres réseau de VirtualBox : rediriger le port 2222 de l'hôte vers le port 22 de la VM. Ensuite on peut se connecter avec :

```
ssh -p 2222 user@127.0.0.1
```

Source : Manuel officiel VirtualBox

2 Préparation du système

2.1 Accès SUDO pour user

2.1.1 Se connecter en ROOT

Appuyer sur CTRL + ALT + F1 pour passer en mode console

Après “server login” entrer “root” puis également dans le champs mot de passe (le mot de passe ne sera pas visible a la saisie)

2.1.2 Donner les droits SUDO au compte user

Entrer la commande suivante :

```
usermod -aG sudo user
```

Explication de la commande :

usermod : User Modify, c'est la commande en elle même, qui permet de modifier un utilisateur

-aG* : Append Group, permet d'indiquer qu'on veut ajouter l'utilisateur a un groupe. Attention a bien taper “aG” et non “G” car sinon l'utilisateur sera retiré de tout ses autres groupes

sudo : indique que l'utilisateur sera ajouté dans le groupe SUDO

user : l'utilisateur a qui on veut rajouter les droits

2.1.3 Se reconnecter et recevoir les droits

Appuyer sur CTRL + ALT + F7 afin de revenir en vue graphique et cliquer sur “Système → Fermer la session de user... → Déconnexion” afin de se déconnecter

Se reconnecter en tant que User et exécuter la commande “groups ” pour vérifier si sudo s'affiche bien, sinon recommencer le processus.

2.1.4 Questions

Question 1 : Comment peut-on savoir à quels groupes appartient l'utilisateur user ?

Pour savoir à quels groupe appartient l'utilisateur *user*, taper **groups** en étant connecté en tant qu'*user* et sinon **groups user** en tant qu'administrateur

Question 2 : Quelle est la différence entre su et sudo ?

`su` permet de se connecter en tant qu'un autre utilisateur, généralement `root` (`su -`), pour exécuter des commandes administrateurs. `sudo` permet d'exécuter une seule commande administrateur, seulement si l'utilisateur qui l'utilise possède les droits SUDO.

2.2 Installation des suppléments invités

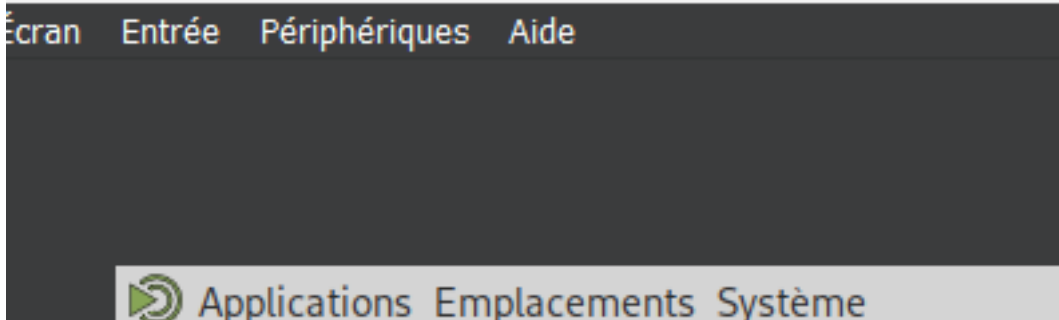


Figure 10: Bouton périphériques

Appuyer sur le bouton Périphériques puis “Insérer l'image CD des Additions invité ...”

```
sudo mount /dev/cdrom /mnt
```

Exécuter cette commande puis entrer le mot de passe demandé.

Exécuter ensuite cette commande :

```
sudo /mnt/VBoxLinuxAdditions.run
```

2.2.1 Questions

Question 1 : Quelle est la version du noyau Linux utilisé par votre VM ?

La version du noyau Linux utilisé par la VM est le 6.12.73. Il est obtenu en faisant la commande `uname -r`. Cette commande renvoie quelque chose comme **6.12.73+deb13-amd64** qui signifie que le noyau linux est en version **6.12.73**, qu'il est adapté pour debian 13 et que c'est pour l'architecture x64.

Source : `man uname`

Question 2 : À quoi servent les suppléments invités ? Donner 2 principales raisons de les installer.

Les suppléments invité sont utilisés pour améliorer l'intégration avec l'ordinateur hôte dans VirtualBox.

Ajout	Description
Meilleure intégration de la souris	La souris peut passer plus facilement et fluidement de l'ordinateur à la VM
Presse-papier partagé	Tout ce que l'on copie de l'ordinateur sera aussi dans le presse-papier de la VM et inversement
Dossiers partagés	Accès plus facile aux dossiers de l'ordinateur depuis la VM
Meilleur support graphique	C'est grâce à cette fonctionnalité que l'on peut modifier la taille de la fenêtre de notre VM et que cette dernière s'adapte totalement

Source : Manuel Officiel VirtualBox

Question 3 : À quoi sert la commande mount (dans notre cas de figure et dans le cas général) ?

La commande mount sert en général à **rattacher un système de fichiers dans l'arborescence du système** pour pouvoir accéder à son contenu. Elle sert en général à rendre accessibles des périphériques de stockage. Dans notre cas elle permet de monter le CD qui contient les suppléments invités de virtualbox pour pouvoir les installer ensuite.

Source : man mount

Source : Manuel Officiel VirtualBox

3 Distribution Debian

3.1 Quelques questions sur Debian

Question 1 : Qu'est-ce que le projet Debian ? D'où vient le nom ?

Debian est une distribution **GNU Linux** qui se veut *libre* et de *qualité supérieure*. Développé entièrement **bénévolement**, la distribution Debian est **stable**, **complète**, **gratuite** tant à l'utilisation qu'à l'amélioration et la redistribution et **active** grâce au travail régulier de plus de **1600 bénévoles**.

Le nom Debian vient de la contraction de **Debra** et **Ian** Murdock, les deux créateurs originels du projet.

Source : Documentation debian, FAQ

3.1.1 La maintenance

Question 2 : Il existe 3 durées de prises en charge de ces versions : la durée minimale, la durée en support Long terme et la durée en support long terme étendue. Quelles sont les durées de ces prises en charge ?

Debian à 3 niveaux de supports différents avec des durées de prises en charge différentes.

Type	Durée	Total Cumulé
Support Standard	~3 ans	3 ans
LTS (Long Term Support)	+2 ans	5ans
ELTS (Extended LTS)	+5 ans	10 ans

Le support est géré par les équipes debian officielle. Le LTS et ELTS sont gérées par des bénévoles.

Source LTS et Debian Stable

Source ELTS

Source Releases

Question 3 : Pendant combien de temps les mises à jour de sécurité sont-elles fournies ?

Actuellement, les versions récentes de Debian reçoivent :

- 3 ans de support standard (mises à jour de sécurité)
- +2 ans avec LTS → 5 ans au total
- +5 ans avec ELTS → jusqu'à 10 ans au total

Les anciennes versions (avant Debian 6) avaient des durées plus courtes car le LTS n'existait pas encore.

Source LTS et Debian Stable

Source ELTS

Source Releases

3.1.2 Nom générique, nom de code et version

Question 4 : Combien de versions au minimum sont activement maintenues par Debian ? Donnez leur nom générique (= les types de distribution).

Debian a toujours au moins trois versions en maintenance active : **stable**, **testing** et **unstable**.

Voici à quoi elles correspondent :

- **Stable** : La distribution stable contient la dernière distribution officiellement publiée de Debian. C'est la version de production de Debian, celle qu'on recommande principalement d'utiliser. Actuellement, c'est **Trixie** (Debian 13).
- **Testing** : Testing est la branche de prévisualisation qui deviendra la prochaine version majeure. Les paquets inclus dans cette branche ont été testés dans unstable mais peuvent ne pas être prêts pour la publication. Elle contient des paquets plus récents que stable mais plus anciens qu'unstable. Actuellement, c'est **Forky** (futur Debian 14).
- **Unstable** : La distribution unstable s'appelle toujours **sid**. Unstable, toujours nommée sid, est le tronc. Les paquets y sont acceptés sans vérification de la distribution dans son ensemble. Cette branche est généralement utilisée par les développeurs qui participent à un projet et ont besoin des dernières bibliothèques.

Source : Debian

Question 5 : Chaque distribution majeure possède un nom de code différent. Par exemple, la version majeure actuelle (Debian 13) se nomme trixie. D'où viennent les noms de code donnés aux distributions ?

Les versions de Debian ont des noms de code tirés de Toy Story (Buzz, Woody, etc.). C'est Bruce Perens, qui était leader du projet et bossait chez Pixar, qui a eu cette idée.

La version unstable s'appelle toujours "Sid" en référence au voisin qui casse les jouets dans le film.

Source : Debian

Source : Debian Doc

Question 6 : L'un des atouts de Debian fut le nombre d'architecture (~ processeurs) officiellement prises en charge. Combien et lesquelles sont prises en charge par la version Trixie ?

Trixie prend en charge 9 architectures officielles :

- amd64
- i386
- arm64
- armel
- armhf
- mips64el
- ppc64el
- s390x
- riscv64

Source : Debian

Source : Wikipedia, Debian Version History

Question 7 : Première version avec un nom de code

Le premier nom de code utilisé pour une version Debian est Buzz (en référence à Buzz l'Éclair de Toy Story). Son numéro de version était Debian 1.1 (et non 1.0, la version 1.0 n'étant jamais sortie). Elle a été annoncée le 17 juin 1996 et contenait 474 paquets. C'était la première version stable officielle portant un nom de code.

Source : Wikipedia, Debian Version History

Question 8 : Dernier nom de code attribué

Le dernier nom de code annoncé à ce jour pour une distribution Debian est **Duke**. Il correspond à la version *Debian 15* et a été annoncé le 22 janvier 2025.

Source : Wiki Debian

4 Automatisation

4.1 Récupérer et préparer les fichiers nécessaires

Afin de mener à bout ce processus vous aurez besoin de : L'application Oracle Virtual Box Les fichiers de configuration et VISO de Moodle mentionnées sur le PDF de la SAE.

Extraire depuis Moodle le fichier zip avec le Viso et les fichiers de configuration pour l'installation et placer les contenus dans le dossier où vous avez fait la VM

4.1.1 Création de la VM

A l'aide de Virtual Box, créer une VM avec ces spécifications :

- Type : Linux
- Version : Debian 64-bit
- Mémoire vive (RAM) : 2048 Mo pour être à l'aise à l'usage.
- Disque dur : 20 Go, ne pas cocher la case "Pre-allocate Full Size"
- Cocher la case "Skip Unattended Installation" pour éviter que Virtualbox réalise des actions non souhaitées.

4.1.2 Configurations

- Remplacer la chaîne @@UUID@@ du fichier S203_Debian13.viso par un identifiant unique universel. Le plus simple est d'exécuter la commande ci-dessous en étant placé dans le même répertoire que votre fichier :

```
sed -i -E "s/(--iprt-iso-maker-file-marker-bourne-sh).*/\1=$(cat /proc/sys/kernel/random/uuid)/" S203-Debian13.viso
```

- Dans le fichier **preseed.cfg** qui se trouve au même endroit que le **viso** on doit ajouter quelques commandes pour automatiser totalement l'installation. Ces commandes vont installer quelques applications dont nous avons besoin comme **git** ou **bash-completion** et ajouter l'utilisateur normal au groupe **sudo**.
 - A la ligne 83 : `tasksel tasksel/first multiselect standard ssh-server mate-desktop`
 - A la ligne 84 : `d-i pkgssel/include string git sudo sqlite3 curl bash-completion fastfetch`
 - A la ligne 56 : `d-i passwd/user-default-groups string audio cdrom video sudo`

Le fichier ressemblera à ceci :

```

78
79 ## Installation meta-paquetages
80 # Tâches à installer (via des méta-paquetages)
81 #   lister les possibilités : tasksel --list-task (en ligne de commande)
82 #   Utiliser au minimum "standard" est une bonne idée
83 tasksel tasksel/first multiselect standard ssh-server mate-desktop
84 d-i pkgssel/include string git sudo sqlite3 curl bash-completion neofetch

```

Figure 11: Ajout des applis.

```

45 ### Ajout des comptes root et user
46 d-i user-setup/allow-password-weak boolean true
47 ## root
48 d-i passwd/root-login boolean true
49 d-i passwd/root-password password root
50 d-i passwd/root-password-again password root
51 ## Utilisateur standard
52 d-i passwd/user-fullname string User
53 d-i passwd/username string user
54 d-i passwd/user-password password user
55 d-i passwd/user-password-again password user
56 d-i passwd/user-default-groups string audio cdrom video sudo

```

Figure 12: Ajout de l'utilisateur au groupe audio, cdrom, video et sudo.

- Aller dans la configuration de la VM et mettre comme **ISO** d'installation le fichier **viso** auquel on a appliqué la commande **sed** précédemment.
- Démarrer la VM et attendre que l'installation se complète

4.1.3 Questions

Question 1 : Quelle est la différence entre `d-i pkgssel/include` et `tasksel tasksel/first` ?

```
tasksel tasksel/first
```

Cette commande permet d'installer des tasks (ensembles de dizaines voire centaines de paquets à la fois)

```
d-i pkgssel/include
```

Cette commande permet d'installer des paquets 1 par 1.

Source : `man tasksel`

Source : Debian

Question 2 : Le `preseed` contient-il des informations sensibles (mots de passe) ? Comment devrait-on protéger ce fichier en production ?

Le `preseed` contient notamment les mots de passe des utilisateurs écrits en clair. Il y aurait plusieurs solutions pour protéger ce fichier en production :

- Ne jamais le mettre sur un GIT public
- Utiliser des mots de passe hachés
- Ne jamais le partager à qui que ce soit
- Restreindre les droits du fichier

Source : le `preseed`

5 Analyse préliminaire de Git et des outils graphiques associés

5.1 Configuration globale de Git

5.1.1 Questions

Question 1 : Quelle est la différence entre Git et les logiciels comme GitHub / GitLab / Forgejo ?

Git est un logiciel de contrôle de version créé par Linus Torvalds en 2005. Il tourne en local sur la machine. Il permet de suivre l'historique des fichiers, revenir en arrière, travailler à plusieurs, etc. C'est un outil en ligne de commande.

GitHub / GitLab / Forgejo sont des plateformes web qui hébergent des dépôts Git en ligne. Elles ajoutent plein de fonctionnalités autour de Git :

- Une interface web pour visualiser le code
- La gestion des issues (tickets de bugs, demandes)
- Les pull/merge requests (proposer des modifications)
- Un système de CI/CD (tests automatiques, déploiement)
- La gestion des droits d'accès, wikis, etc.

Source : Git

Source : GitHub

Question 2 : Qu'est-ce qu'un dépôt Git ? Où sont stockées les données d'un dépôt local ?

Un dépôt (repository) est un répertoire dont Git suit l'historique des modifications. On initialise un dépôt avec `git init`, ce qui crée un sous-répertoire caché `.git/` à la racine du projet.

C'est dans ce dossier `.git/` que Git stocke l'intégralité de l'historique.

Source : Git

Question 3 : Expliquez la différence entre un commit, une branche et un tag.

Un commit est un instantané de l'état du projet à un moment donné. Il contient un identifiant unique, un auteur, une date, un message descriptif et une référence vers le ou les commits parents.

Une branche est un pointeur mobile vers un commit. À chaque nouveau commit effectué sur cette branche, le pointeur avance automatiquement. Les branches permettent de développer des fonctionnalités en parallèle sans impacter le reste du projet. La branche par défaut est généralement `main`.

Un tag est un pointeur fixe vers un commit précis. Contrairement à une branche, il ne se déplace jamais. On l'utilise pour marquer des jalons importants, typiquement des numéros de version.

Source : Git

Question 4 : Qu'est-ce qu'un dépôt distant (remote) ? Comment lister les remotes configurés ?

Un remote est une version du dépôt hébergée sur un serveur distant (GitHub, GitLab, serveur de l'IUT...). Il permet de synchroniser son travail local avec d'autres collaborateurs.

Lors d'un `git clone`, Git configure automatiquement un remote nommé `origin`, qui pointe vers l'URL source.

Pour lister les remotes configurées :

```
git remote -v
```

Source : Git

Question 5 : Quelle est la différence entre git pull et git fetch ?

`git fetch` télécharge les modifications depuis le dépôt distant, mais ne modifie pas votre branche de travail locale. Les données récupérées sont stockées dans les références distantes (ex : origin/main). Cela permet de consulter les changements avant de les intégrer.

`git pull` effectue un `git fetch` suivi d'un `git merge`. Il récupère les nouveautés distantes et les fusionne immédiatement dans la branche courante

Source : Git Source : Git

5.2 Protocoles réseau pour Git

5.2.1 Questions

Question 1 : Quels sont les protocoles supportés par Git pour accéder à un dépôt ?

Git supporte quatre protocoles pour accéder à un dépôt distant :

- **HTTPS** Le plus courant aujourd'hui. Il fonctionne partout, passe les pare-feux sans problème et chiffre les > échanges.

```
https://gitlab.univ-lille.fr/user/projet.git
```

L'authentification se fait par identifiant/mot de passe ou par token d'accès personnel.

- **SSH** Le protocole privilégié pour les développeurs. Il utilise un système de clés asymétriques (clé publique/privée), ce qui évite de saisir un mot de passe à chaque opération.

```
git@gitlab.univ-lille.fr:user/projet.git
```

- **Protocole Git natif** Un protocole léger et rapide, mais sans aucune authentification ni chiffrement. Il écoute sur un démon dédié (git-daemon). On le reconnaît au préfixe git://.

```
git://exemple.com/projet.git
```

Il est adapté uniquement à la lecture publique de dépôts ouverts.

- **Protocole local (file)** Utilisé quand le dépôt distant se trouve sur le même système de fichiers.

```
git clone file:///srv/git/projet.git
```

C'est le cas le plus simple, mais rarement utilisé en pratique pour de la collaboration.

Source : Git

Question 2 : Sur quels ports réseau fonctionnent ces protocoles ?

Protocole	Port par défaut	Chiffrement	Authentification
HTTPS	443 (TCP)	Oui (TLS)	Mot de passe / token
SSH	22 (TCP)	Oui (SSH)	Clé publique / mot de passe
Git natif	9418 (TCP)	Non	Aucune
Local (file)	Aucun (pas de réseau)	N/A	Droits du système de fichiers

Source : Git

Question 3 Comment configurer l'authentification SSH pour Git ?

On génère une paire de clés (une privée qu'on garde, une publique qu'on donne au serveur), et ensuite Git utilise SSH pour s'authentifier automatiquement.

Etape 1 : vérifier si une clé existe déjà

```
ls -al ~/.ssh/
```

Si des fichiers `id_ed25519` et `id_ed25519.pub` (ou `id_rsa` / `id_rsa.pub`) sont déjà présents, il y a déjà une paire de clés. Passer directement à l'étape 3.

Etape 2 : Générer une paire de clés

```
ssh-keygen -t ed25519 -C "prenom.nom@etu.univ-lille.fr"
```

Le terminal demande :

```
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/user/.ssh/id_ed25519):
```

Appuyer sur Entrée pour accepter le chemin par défaut. Il demande ensuite une passphrase, c'est le mot de passe qui protège la clé privée.

Résultat : 2 fichiers sont créés : - `~/.ssh/id_ed25519` -> clé privée - `~/.ssh/id_ed25519.pub` -> clé publique

Il est recommandé d'utiliser Ed25519 plutôt que RSA car c'est un algorithme plus moderne, plus rapide et avec des clés plus courtes à sécurité équivalente.

Etape 3 : copier la clé publique

```
cat ~/.ssh/id_ed25519.pub
```

Copier toute la ligne de résultat.

Etape 4 : Ajouter au GitLab

1. Se connecter au gitlab
2. Cliquer sur l'avatar -> Préférences
3. Dans le menu de gauche -> Clés SSH
4. Coller la clé publique dans le champs key
5. Donner un titre descriptif
6. Cliquer sur ajouter la clé

Etape 5 : Vérifier que tout fonctionne

```
ssh -T git@gitlab.univ-lille.fr
```

Si tout est bien configuré :

```
Welcome to GitLab, @nom.prenom!
```

Etape 6 : Utiliser SSH pour cloner

Désormais il y a la possibilité de cloner sans mot de passe via SSH :

```
git clone git@gitlab.univ-lille.fr:nom-prenom/projet.git
```

Source : Git Source : GitLab

5.3 Manipulation pratique de Git

5.3.1 Questions

Question 1 : Créez un nouveau dépôt Git local. Quelle commande utilisez-vous ? Que se passe-t-il ?

```
git init
```

Cette commande crée le sous-répertoire `.git/` qui contient toute la structure interne du dépôt : `objects/`, `refs/`, `HEAD`, `config`, etc. Le répertoire de travail est désormais suivi par Git, mais aucun fichier n'est encore versionné — le dépôt est vide, sans aucun commit.

Source : Git

Question 2 : Ajoutez plusieurs fichiers, faites au moins 3 commits. Utilisez `git log` pour visualiser l'historique

```
git log --oneline
```

Resultat :

```
c3a1f9e add file 3
b7d2e4a add file 2
a1f0c3b add file 1
```

Chaque ligne correspond à un commit, avec son hash abrégé et son message. Les commits apparaissent du plus récent au plus ancien.

L'option `--oneline` est un raccourci pour un affichage compact. Sans cette option, `git log` affiche pour chaque commit : le hash complet, l'auteur, la date et le message.

Source : Git

Question 3 : Créez une nouvelle branche, faites des modifications, puis fusionnez (merge) cette branche avec la branche principale. Expliquez le processus.

Pour créer la nouvelle branche :

```
git branch branch2
git checkout branch2
```

Après modifications et commit, la branche `branch2` a un commit d'avance sur `main`.

Revenir sur `main` et fusionner :

```
git checkout main
git merge branch2
```

Ici, Git réalise un `fast-forward` : comme `main` n'a pas bougé entre-temps, il se contente d'avancer le pointeur de `main` jusqu'au dernier commit de la branche. Aucun commit de fusion supplémentaire n'est nécessaire.

Nettoyage

```
git branch -d branch2
```

La branche a rempli son rôle, on peut la supprimer. L'historique des commits qu'elle contenait est conservé dans `main`.

Source : git

Question 4 : Qu'est-ce qu'un conflit de fusion (merge conflict) ? Provoquez volontairement un conflit et montrez comment le résoudre

Le fichier en conflit ressemble à ça :

```
<<<<<< HEAD
ligne1 main
=====
ligne1 conflit
>>>>>> test-conflit
```

- Entre «<<< HEAD et ===== : la version de main (branche courante)
- Entre ===== et »>>>> test-conflit : la version de la branche qu'on fusionne

Pour résoudre le conflit il suffit de modifier la ligne en question, retirer les «<<<, ===== et »>>>> puis ;

```
git add file1.txt
git commit -m "resolving conflict"
```

Source : Git

Question 5 : Utilisez git diff pour comparer deux commits. Expliquez la sortie.

```
diff --git a/file1.txt b/file1.txt
index 5a2c8f3..b3d7e1a 100644
--- a/file1.txt
+++ b/file1.txt
@@ -1 +1 @@
-fafjalkfjaljflf
+testcommit1
```

Element	Signification
diff --git a/file1.txt b/file1.txt	Fichier comparé entre les deux commits
--- a/file1.txt	Version du fichier dans le premier commit
+++ b/file1.txt	Version du fichier dans le second commit
@@ -1 +1 @@	Position dans le fichier : ligne 1, dans les deux versions
-fafjalkfjaljflf	Ligne supprimée (préfixe -, version ancienne)
+testcommit1	Ligne ajoutée (préfixe +, version nouvelle)

Les lignes sans préfixe (ni + ni -) sont les lignes inchangées, affichées pour donner du contexte.

Source : Git

Question 6 : Qu'est-ce que le fichier .gitignore ? Créez-en un pour ignorer les fichiers de compilation Java (.class,.jar).

Le fichier .gitignore indique à Git quels fichiers ou répertoires ne doivent pas être suivis. Git les ignore lors des git add et ne les inclut pas dans les commits. C'est utile pour exclure les fichiers générés automatiquement (compilation, logs, dépendances...) qui n'ont pas leur place dans le dépôt.

```
.gitignore
*.class
*.jar
```

Source : Git

5.4 Travail collaboratif

5.4.1 Questions

Question 1 : Qu'est-ce qu'une pull request (ou merge request dans GitLab) ? À quoi sert-elle ?

Une merge request est une demande formelle pour intégrer les modifications d'une branche dans une autre. C'est un mécanisme propre aux plateformes comme GitLab/GitHub/Forgejo, ça n'existe pas dans Git lui-même.

Plutôt que de fusionner directement sa branche dans main avec `git merge`, on passe par la plateforme web pour demander cette fusion. Cela ouvre un espace de discussion où l'équipe peut :

- Relire le code ligne par ligne
- Commenter des passages précis, poser des questions, suggérer des améliorations
- Demander des corrections avant d'accepter la fusion
- Lancer des tests automatiques (CI/CD) pour vérifier que rien n'est cassé
- Valider la fusion quand tout est satisfaisant

Source : GitLab Source : Git

Question 2 : Testez la commande `git blame`. À quoi sert-elle ?

`git blame` affiche chaque ligne d'un fichier avec le commit, l'auteur et la date de sa dernière modification. En résumé : qui a écrit quoi, et quand.

Ca peut servir à :

- Comprendre l'historique : quand on tombe sur une ligne de code bizarre, on sait immédiatement qui l'a écrite et dans quel commit, ce qui permet de retrouver le contexte (message du commit, merge request associée...) -Trouver le responsable : si une ligne introduit un bug, `git blame` identifie l'auteur à contacter -Retracer l'évolution : on peut suivre comment un fichier a évolué au fil du temps, commit par commit

Source : Git Source : Git

6 Les interfaces graphiques pour Git

6.1 Questions

Question 1 : Qu'est ce que le logiciel *gitk* ? Comment se lance-t-il ?

`gitk` est un visualiseur d'historique graphique pour Git. Il permet de naviguer dans l'historique des commits de manière visuelle, sous forme d'un graphe de branches et de commits.

Il affiche :

- Le graphe des commits avec les branches et les fusions (à gauche)
- Le détail du commit sélectionné : auteur, date, message (en haut à droite)
- Le diff des modifications introduites par ce commit (en bas à droite)
- La liste des fichiers modifiés

Il se lance depuis un terminal, à l'intérieur d'un dépôt Git :

```
gitk
```

Options utiles :

```
#Voir toutes les branches
gitk --all

#Voir uniquement les 20 derniers commits
gitk -20
```

Source : Git

Question 2 : Qu'est ce que le logiciel *git-gui* ? Comment se lance-t-il ?

git gui est une interface graphique pour effectuer des opérations Git courantes. Contrairement à gitk qui est en lecture seule, git gui permet d'agir sur le dépôt.

Il permet notamment de :

- Visualiser les fichiers modifiés, indexés (staged) et non suivis
- Indexer des fichiers (équivalent de git add), fichier par fichier ou ligne par ligne
- Créer des commits avec un message, directement depuis l'interface
- Modifier le dernier commit (amend)
- Pousser vers un dépôt distant (push)
- Créer et gérer des branches et des tags

Il se lance depuis un terminal, à l'intérieur d'un dépôt Git :

```
git gui
```

Source : Git Source : Git

7 Installons autre chose et comparons

7.1 Questions

Question 1 : Pourquoi avez-vous choisi ce logiciel ?

Nous avons choisi GitKraken car il est toujours gratuit pour une utilisation avec des dépôts locaux et des dépôts public hébergés dans le cloud. Il a une interface graphique assez intuitive et est multiplateforme (Windows, mac, linux).

Source : GitKraken

Question 2 : Comment l'avez vous installé ?

Sur debian 13, nous avons téléchargé le paquet .deb et l'avons installé comme suit :

```
wget https://release.gitkraken.com/linux/gitkraken-amd64.deb
sudo apt install ./gitkraken-amd64.deb
```

Sur linux, il se lance en tapant `gitkraken` dans un terminal.

Source : GitKraken

Question 3 : Comparez-le aux outils inclus avec git (et installés précédemment) ainsi qu'avec ce qui serait fait en ligne de commande pure : fonctionnalités avantages, inconvénients...

Critère	Ligne de commande	gitk / git gui	GitKraken
Prix	Gratuit	Gratuit	Gratuit (dépôts locaux/publics)
Visualisation de l'historique	git log --graph (texte brut)	gitk : graphe basique	Graphe interactif coloré, cliquable

Critère	Ligne de commande	gitk / git gui	GitKraken
Faire un commit	<code>git add + git commit</code>	git gui : interface fonctionnelle mais austère	Interface glisser-déposer, indexation par ligne
Résolution de conflits	Édition manuelle des marqueurs	Pas d'outil intégré	Éditeur de merge côte à côte
Diff entre versions	<code>git diff</code> (texte)	gitk : diff basique	Diff côte à côte avec coloration syntaxique
Gestion de branches	<code>git branch, git merge</code>	Basique	Drag-and-drop pour merge
Intégrations	Aucune	Aucune	GitHub, GitLab, ...
Rapidité d'exécution	La plus rapide	Rapide	Plus lourde
Apprentissage	Courbe raide	Intermédiaire	Facile pour les débutants
Disponibilité	Partout, toujours	Nécessite un affichage graphique	Nécessite un affichage graphique + installation
Scriptable / automatisable	Oui	Non	Non

Avantages de GitKraken :

- Interface moderne qui rend les concepts Git (branches, merge, rebase) visuellement compréhensibles
- Intégrations avec les plateformes (GitLab, GitHub...)
- Résolution de conflits intégrée et visuelle

Inconvénients :

- La version gratuite de GitKraken ne permet pas de travailler avec des dépôts privés hébergés sur des plateformes (il faut payer pour ça)
- Application propriétaire, contrairement à gitk/git gui qui sont open source
- GitKraken est une application lourde basée sur Electron, tandis que gitk et git-gui sont des outils très légers installés en quelques secondes.

Source : Git Source : GitKraken Source : GitKraken

Question 4 + 5 : Réalisez les opérations suivantes en ligne de commande puis avec l'outil graphique de votre choix, et comparez : visualiser l'historique des commits — créer une nouvelle branche — comparer deux versions d'un fichier (diff) — résoudre un conflit de fusion. + Quelles opérations sont plus rapides en ligne de commande ? Lesquelles sont plus claires avec l'interface graphique ?

1 - Visualiser l'historique des commits

Ligne de commande :

```
git log --oneline --graph --all
```

Résultat : texte en brut avec des caractères ASCII pour dessiner les branches

GitKraken :

Ouvrir le dépôt et le graphe s'affiche automatiquement. Chaque branche a une couleur, chaque commit est cliquable, on voit les auteurs, les dates, les tags d'un coup d'œil.

Verdict : GitKraken est nettement plus clair. Sur un projet avec beaucoup de branches, le graphe ASCII devient illisible alors que GitKraken reste explorable.

2 - Créer une nouvelle branche

Ligne de commande :

```
git branch branch-name
git checkout branch-name
```

Deux commandes, execution instantanées

GitKraken :

Clic droit sur un commit -> Create Branch here -> nommer -> valider

Verdict : La ligne de commande est plus rapide

3 - Comparer deux versions d'un fichier(diff)

Ligne de commande :

```
git diff abc123 def456 -- fichier.txt
```

Résultat en texte avec les préfixes + et -.

GitKraken :

On clique sur un commit, puis sur le fichier modifié. Le diff s'affiche côte à côte avec coloration syntaxique.

Verdict : GitKraken est beaucoup plus lisible, surtout sur les gros diffs.

4 - Résoudre un conflit de fusion

En ligne de commande :

Aller à la question qui y répond déjà...

GitKraken :

Quand un conflit survient, GitKraken ouvre un éditeur de merge avec trois panneaux : la version de gauche (branche courante), la version de droite (branche entrante) et le résultat en bas. On coche ce qu'on veut garder, on voit le résultat en temps réel.

Verdict : GitKraken est largement supérieur pour les conflits. C'est plus visuel, moins risqué, et on ne peut pas oublier un marqueur <<<<<< dans le code.

Synthèse

Opération	Plus rapide en...	Plus clair en...
Visualiser l'historique	Équivalent	GitKraken
Créer une branche	Ligne de commande	Équivalent
Diff entre versions	Ligne de commande	GitKraken
Résoudre un conflit	Équivalent	GitKraken
Opérations en série (add, commit, push)	Ligne de commande	Équivalent
Exploration d'un projet inconnu	GitKraken	GitKraken

Source : Git

Source : GitKraken

Source : GitKraken

Question 6 : Dans un contexte professionnel, quel outil privilégieriez-vous et pourquoi ?

Dans un contexte professionnel l'idéal serait de combiner les deux :

- Ligne de commande pour les opérations quotidiennes rapides (add, commit, push, pull, creation de branches). Elle est disponible partout, y compris sur les serveurs sans interface graphique.
- Un outil graphique comme GitKraken pour la visualisation de l'historique sur des projets complexes, la résolution de conflits de fusion, et l'exploration d'un projet qu'on ne connaît pas encore. L'interface visuelle réduit les erreurs dans les situations délicates.

Source : Git

Source : GitKraken

Source : GitKraken

8 Préliminaire : redirection de ports

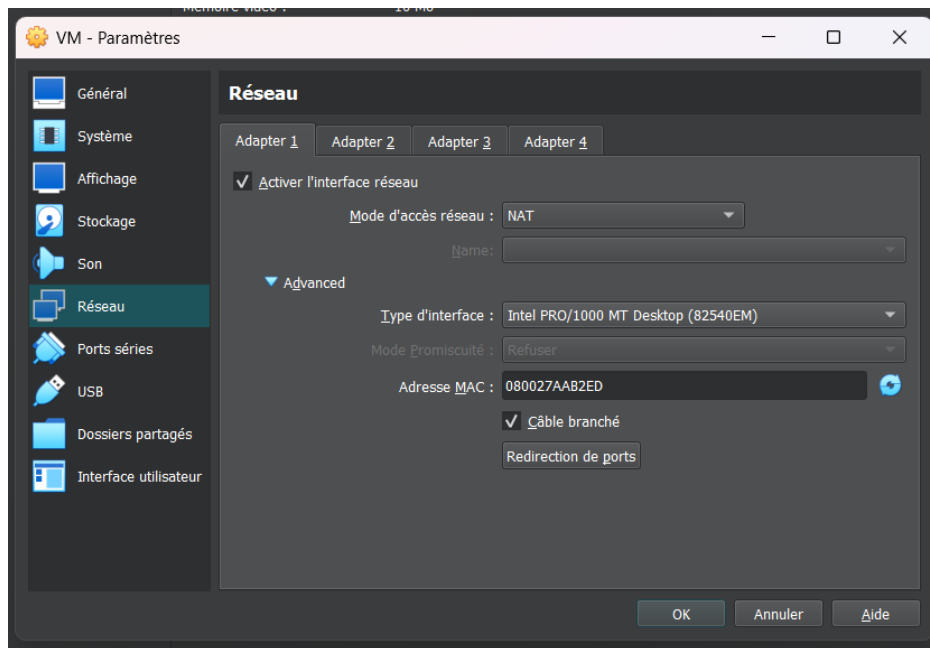


Figure 13: Accès à la configuration des redirections

En mode NAT, la VM est invisible depuis l'hôte. La redirection de port permet d'accéder au service Forgejo (qui écoute sur le port 3000 de la VM) en allant sur `http://localhost:3000` depuis le navigateur de la machine hôte.

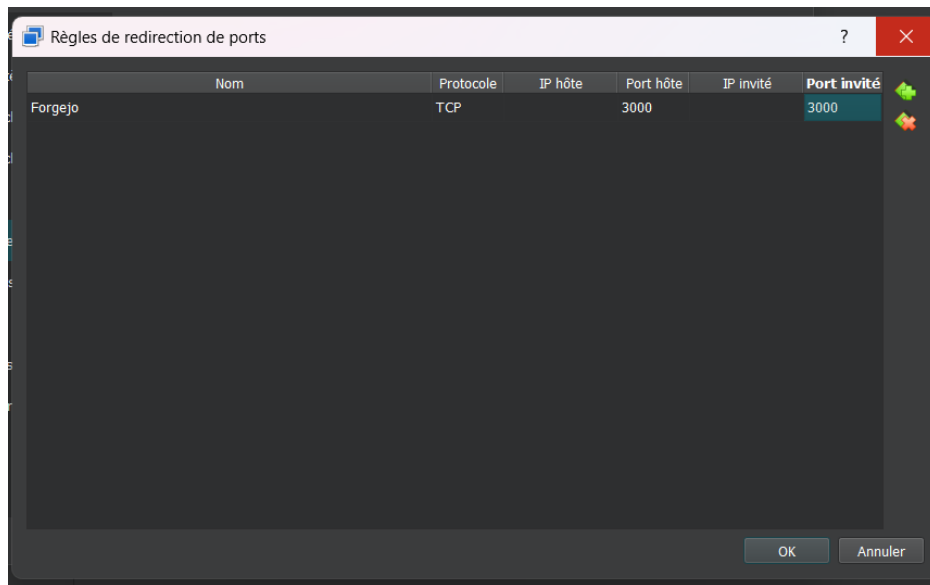


Figure 14: Configuration de la redirection de ForgeJo

8.1 Questions

Question 1 : Qu'est-ce que Forgejo ?

Forgejo est une forge logicielle légère et auto-hébergée. C'est une plateforme similaire à GitHub ou GitLab, mais conçue pour être installée sur son propre serveur. Elle permet par exemple d'héberger des dépôts Git.

Question 2 : Qu'est-ce qu'un fork (dans le domaine du développement logiciel bien entendu) ?

Un fork, c'est quand un développeur (ou un groupe) prend le code source d'un logiciel existant et le copie pour en faire un projet indépendant, qui évolue ensuite séparément. C'est rendu possible par les licences open source.

Question 3 : Forgejo est le résultat d'une chaîne de forks. Retracer cette histoire : De quel logiciel est-il issu ? Pourquoi ce fork a-t-il eu lieu ? Les logiciels d'origine existent-ils encore ?

-> **Gogs** (2014)

-> **Gitea** (2016) - Fork de *Gogs*

-> **Forgejo** (2022) - Fork de *Gitea*

Gogs a été créé en 2014 par un développeur chinois. C'est une forge Git légère écrite en Go. Le projet existe encore aujourd'hui.

Gitea est né en 2016 comme fork de Gogs, car la communauté trouvait que le développement de Gogs était trop lent et que le mainteneur principal ne prenait pas assez en compte les contributions extérieures. Gitea existe encore.

Forgejo est né fin 2022 comme fork de Gitea, après que Gitea a été transféré à une entreprise à but lucratif.

Question 4 : Qu'est-ce qui différencie la gouvernance de Forgejo de celle de Gitea ?

Gitea est contrôlé par une entreprise privée. Les décisions stratégiques sont prises par cette société. Forgejo est gouverné par une communauté, sous l'égide de l'association à but non lucratif (Codeberg e.V.). Les

décisions se prennent collectivement et de manière transparente. C'est d'ailleurs la raison principale du fork : le transfert de Gitea à une entreprise sans consultation de la communauté.

Source : Forgejo - Compare to Gitea

Source : Forgejo - Faq

Source : Forgejo

9 Installation de Forgejo

9.1 Installation de Forgejo

9.1.1 Installation des dépendances

```
sudo apt install git git-lfs
```

Git est nécessaire car Forgejo gère des dépôts Git. `git-lfs` permet la gestion des fichiers volumineux.

9.1.2 Télécharger le binaire

Récupérer le binaire depuis la page de téléchargement officielle pour l'architecture `linux-amd64`, puis vérifier la signature GPG comme indiqué.

```
sudo cp forgejo-x.y.z-linux-amd64 /usr/local/bin/forgejo
sudo chmod 755 /usr/local/bin/forgejo
```

Ces commandes servent à copier le binaire dans `/usr/local/bin/` (qui est le répertoire standard pour les binaires installés manuellement) et ensuite à le rendre exécutable par tous (755).

9.1.3 Créer l'utilisateur système Git

```
sudo adduser --system --shell /bin/bash --gecos 'Git Version Control' --group --disabled-password --home /
```

Détail des options de cette commande :

Option	Rôle
<code>--system</code>	Crée un utilisateur système
<code>--shell /bin/bash</code>	Lui attribue un shell (nécessaire pour les opérations Git via SSH)
<code>--gecos 'Git Version Control'</code>	Description de l'utilisateur
<code>--group</code>	Crée aussi un groupe du même nom
<code>--disabled-password</code>	Pas de mot de passe (connexion par mot de passe impossible)
<code>--home /home/git</code>	Répertoire personnel
<code>git</code>	Nom de l'utilisateur

9.1.4 Créer les répertoires

```
# Répertoire des données
sudo mkdir /var/lib/forgejo
sudo chown git:git /var/lib/forgejo && sudo chmod 750 /var/lib/forgejo
```

```
# Répertoire de configuration
sudo mkdir /etc/forgejo
sudo chown root:git /etc/forgejo && sudo chmod 770 /etc/forgejo
```

Répertoire	Contenu	Propriétaire	Droits	Pourquoi
/var/lib/forgejo	Données (dépôts, BDD SQLite, LFS...)	git:git	750	Forgejo (qui tourne en tant que git) doit lire et écrire Temporairement en écriture pour git car Forgejo doit créer app.ini lors de la config initiale
/etc/forgejo	Configuration (app.ini)	root:git	770	

9.1.5 SQLite

Aucune configuration de base n'est nécessaire avec sqlite.

9.1.6 Installer le service systemd

```
sudo wget -O /etc/systemd/system/forgejo.service https://codeberg.org/forgejo/forgejo/raw/branch/forgejo/c
```

On télécharge le fichier de service fourni par le projet Forgejo lui-même depuis leur dépôt officiel. Avec SQLite, aucune modification du fichier n'est nécessaire.

Puis :

```
sudo systemctl daemon-reload
sudo systemctl enable forgejo.service
sudo systemctl start forgejo.service
```

- **daemon-reload** : recharge la configuration de systemd
- **enable** : active le démarrage automatique
- **start** : démarre le service maintenant

9.1.7 Configuration initiale via le web

Ouvrir <http://localhost:3000> dans le navigateur de la machine hôte (grâce à la redirection de port).

Configuration initiale

Si vous exécutez Forgejo dans Docker, veuillez lire la [documentation](#) avant de modifier les paramètres.

Paramètres de la base de données

Forgejo nécessite MySQL, PostgreSQL, SQLite3 ou TiDB (avec le protocole MySQL).

Type de base de données * MySQL

Hôte * 127.0.0.1:3306

Nom d'utilisateur * forgejo

Mot de passe *

Nom de base de données * forgejo

Configuration générale

Titre du site * Forgejo
Entrez ici le nom de votre instance. Il sera affiché sur chaque page.

Slogan de l'instance Beyond coding. We Forge.
Entrez le slogan de l'instance ici. En son absence il sera désactivé.

Emplacement racine des dépôts * /var/lib/forgejo/data/forgejo-repositories
Les dépôts Git distants seront stockés dans ce répertoire.

Répertoire racine Git LFS /var/lib/forgejo/data/lfs
Les fichiers suivis par Git LFS seront stockés dans ce dossier. Laissez vide pour désactiver LFS.

Figure 15: Ecran de configuration initiale

Laisser tout les paramètres par défaut à l'exception de **Type de base de données** qu'il faut mettre en **SQLite3**

Ensuite, tout en bas dans *Paramètres de compte administrateur*, entrer les valeurs suivantes : - nom : **forgejo** - password : **forgejosae** (entrer un mot de passe sécurisé de préférence) - email : **git@localhost**

Cliquer ensuite sur Install Forgejo. On se retrouve sur ce tableau de bord :

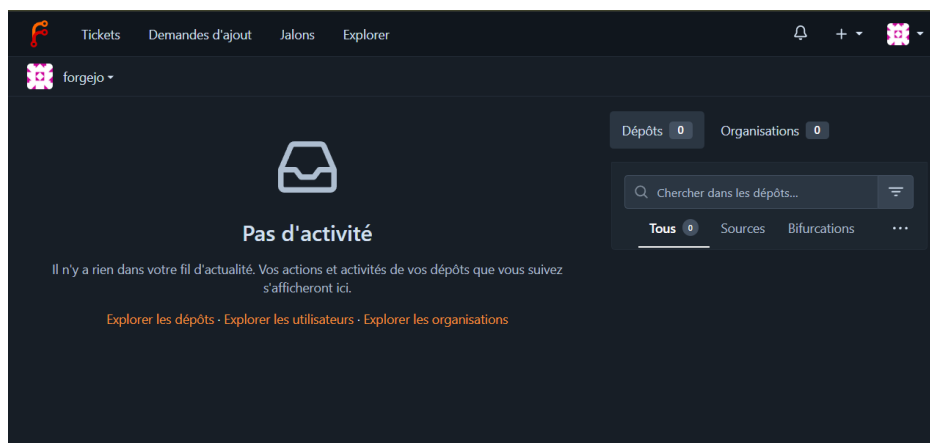


Figure 16: Tableau de bord Forgejo

9.1.8 Verouiller la configuration

```
sudo systemctl stop forgejo.service

sudo chmod 750 /etc/forgejo
sudo chmod 640 /etc/forgejo/app.ini

sudo systemctl start forgejo.service
```

Maintenant que la configuration initiale est terminée, Forgejo n'a plus besoin d'écrire dans `/etc/forgejo`. Passer de 770 à 750 (le groupe git perd le droit d'écriture sur le dossier) et `app.ini` passe en 640 (lisible par git via le groupe, mais plus modifiable).

9.2 Architecture et protocoles réseaux

9.2.1 Questions

Question 1 : Par défaut, Forgejo écoute sur le port 3000. Pourquoi pas le port 80 (port standard HTTP) ?

Sous Linux, les ports inférieurs à 1024 sont des ports privilégiés : seul root peut y faire écouter un service. Forgejo tourne sous l'utilisateur git (un utilisateur non privilégié), donc il ne peut pas écouter sur le port 80.

La doc officielle mentionne d'ailleurs qu'on peut changer le port dans `app.ini` via `HTTP_PORT = 80`, mais ce n'est pas recommandé sans reverse proxy.

Source : `man 7 ip`

Source : Forgejo Docs - Reverse Proxy

9.3 Base de donnée

9.3.1 Questions

Question 1 : Pourquoi nous avons-vous fait choisir SQLite plutôt que PostgreSQL ou MySQL/MariaDB pour ce projet ?

SQLite est une base de données embarquée dans un seul fichier, sans serveur à installer. La doc officielle précise que pour SQLite, il n'y a rien à configurer.

PostgreSQL ou MariaDB nécessiteraient d'installer et configurer un serveur de BDD supplémentaire.

La doc officielle indique que SQLite est adapté pour au moins 10 utilisateurs.

Question 2 : Où se trouve le fichier de base de données SQLite de Forgejo ? Qui en est le propriétaire ?

Le fichier se trouve dans `/var/lib/forgejo/data/`. Le propriétaire est `git:git`, puisque c'est l'utilisateur sous lequel tourne Forgejo, et c'est lui qui doit lire/écrire dans la base.

Source : Forgejo Docs - Installation from binary

Source : SQLite

9.4 Gestion du service Systemd

9.4.1 Questions

Question 1 : Qu'est-ce qu'un service systemd ? À quoi sert-il ?

C'est un programme géré par systemd (le gestionnaire de services de Linux). Systemd peut le démarrer, l'arrêter, le relancer automatiquement en cas de crash, et le lancer au démarrage du système.

Question 2 : Vous avez créé un fichier de service pour Forgejo. Où est-il situé ? Ouvrez-le et expliquez les sections principales : [Unit], [Service], [Install]

Le fichier est situé à `/etc/systemd/system/forgejo.service` (téléchargé depuis le dépôt officiel de Forgejo à l'étape d'installation).

Sections principales :

Section	Role	Contenu important
[Unit]	Décrit le service et ses dépendances	<i>Description=Forgejo</i> , <i>After=network.target</i> → démarre après le réseau
[Service]	Comment exécuter le service	<i>User=git</i> → tourne sous l'utilisateur git, <i>ExecStart=/usr/local/bin/forgejo</i> → commande de lancement, <i>WorkingDirectory=/var/lib/forgejo</i> → répertoire de travail
[Install]	Quand démarrer automatiquement	<i>WantedBy=multi-user.target</i> → se lance en mode multi-utilisateur (le mode normal du système)

Question 3 : Quelle est la différence entre `systemctl start`, `systemctl enable` et `systemctl restart` ?

Commande	Effet
<code>systemctl start forgejo</code>	Démarré immédiatement, ne survit pas au reboot
<code>systemctl enable forgejo</code>	Active le démarrage automatique au boot, ne démarre pas immédiatement
<code>systemctl restart forgejo</code>	Arrête puis redémarre (utile après modification de <i>app.ini</i>)

Question 4 : Comment visualiser les logs du service Forgejo avec `journalctl` ?

```
# Toutes les logs du service
sudo journalctl -u forgejo.service

# Les 100 dernières lignes (comme le suggère la doc officielle pour diagnostiquer)
sudo journalctl -n 100 --unit forgejo.service

# Suivre en temps réel
sudo journalctl -u forgejo.service -f
```

Source : freedesktop

Source : `man systemctl`

Source : `man journalctl`

Source : `man systemd`

9.5 Sécurité

9.5.1 Questions

Question 1 : Examinez les permissions du répertoire `/etc/forgejo` et du fichier `app.ini`. Pourquoi ces permissions sont-elles importantes ?

Commandes utilisées pour vérifier les permissions :

```
ls -ld /etc/forgejo
ls -la /etc/forgejo/app.ini
```

`app.ini` contient des informations sensibles (clé secrète interne, configuration de la base de données, tokens...). Ces permissions garantissent que seuls root (lecture/écriture) et le service Forgejo via le groupe git (lecture seule) y ont accès. Aucun autre utilisateur du système ne peut les consulter.

Question 2 : Le serveur SSH de Forgejo écoute sur quel port ? Pourquoi ce choix ?

Par défaut, Forgejo utilise le serveur SSH du système sur le port 22. Vérifier dans `app.ini` la section `[server]`, paramètre `SSH_PORT`.

Le choix du port 22 est logique car c'est le port SSH standard.

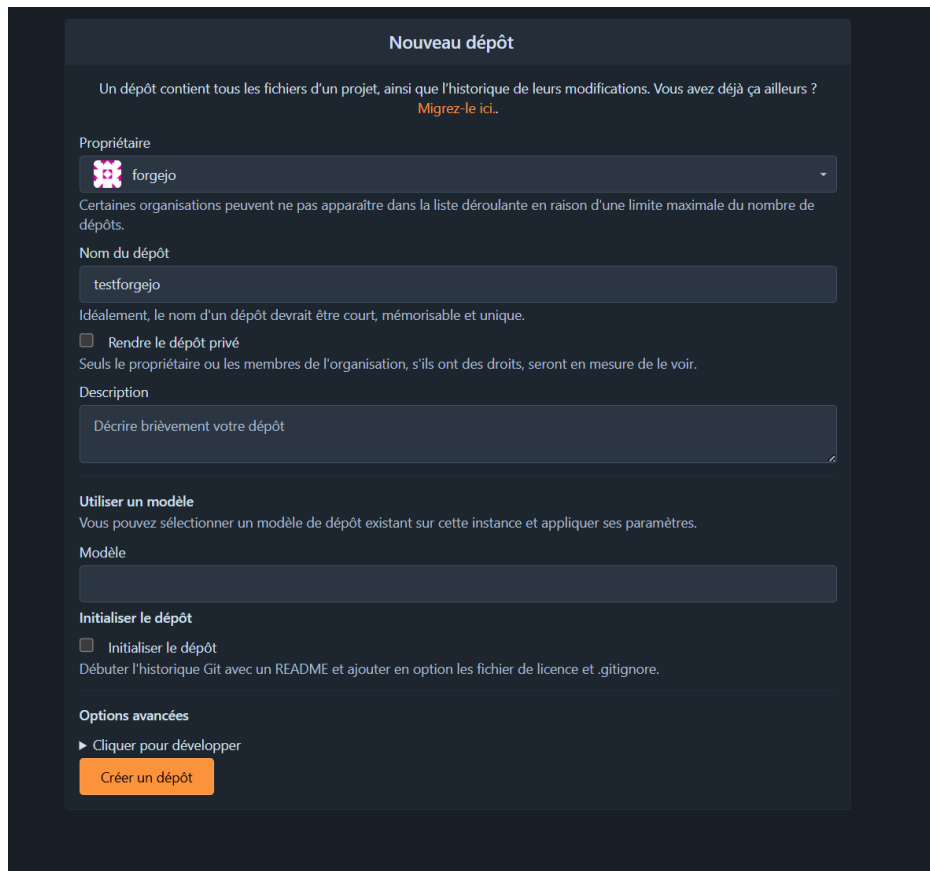
Question 3 : Si votre serveur Forgejo était accessible depuis Internet, quelles mesures de sécurité supplémentaires devriez-vous prendre ?

- HTTPS avec certificat TLS (Transport Layer Security)
- Pare-feu : n'ouvrir que les ports strictement nécessaires
- Mots de passe forts et 2FA (authentification à deux facteurs)
- Mises à jour régulières de Forgejo et du système
- Sauvegardes régulières de la base SQLite et des dépôts

Source : Forgejo Docs - Installation

9.6 Tests d'utilisation

9.6.1 Création d'un dépôt depuis l'interface web



Nouveau dépôt

Un dépôt contient tous les fichiers d'un projet, ainsi que l'historique de leurs modifications. Vous avez déjà ça ailleurs ? [Migrez-le ici.](#)

Propriétaire
forgejo

Certains organisations peuvent ne pas apparaître dans la liste déroulante en raison d'une limite maximale du nombre de dépôts.

Nom du dépôt
testforgejo

Idéalement, le nom d'un dépôt devrait être court, mémorisable et unique.

☐ Rendre le dépôt privé
Seuls le propriétaire ou les membres de l'organisation, s'ils ont des droits, seront en mesure de le voir.

Description
Décrire brièvement votre dépôt

Utiliser un modèle
Vous pouvez sélectionner un modèle de dépôt existant sur cette instance et appliquer ses paramètres.

Modèle

Initialiser le dépôt
☐ Initialiser le dépôt
Débuter l'historique Git avec un README et ajouter en option les fichiers de licence et .gitignore.

Options avancées
► Cliquer pour développer

Créer un dépôt

Figure 17: Création d'un dépôt depuis l'interface web

9.6.2 Ajout des rapports

Créer un depot PRIVE sur l'interface web, sans initialiser le depot.

Cloner le gitlab sur la VM, puis effectuer ces commandes :

```
cd /chemin_vers_le_clone  
  
git remote add forgejo http://localhost:3000/forgejo/rapport-sae203.git  
  
git push forgejo main
```

9.6.3 Les codes sources des TPs

Créer un depot privé sur l'interface web puis l'initialiser comme le rapport mais avec les tp. Sur l'interface web, créer des comptes pour tout les utilisateurs :

ID	Nom d'utilisateur	Courriel	Activé	Restreint	2FA	Créés	Dernière connexion
1	forgejo Administrateur	git@localhost	✓	×	×	27 mars 2026	Jamais connecté
2	lucas	lucas@localhost	✓	×	×	27 mars 2026	Jamais connecté
3	paul	paul@localhost	✓	×	×	27 mars 2026	Jamais connecté

Figure 18: Liste des comptes utilisateur

Ajouter ensuite les utilisateurs au depot (avec des restrictions différentes)



Figure 19: Liste des contributeurs du dépôt

Résultats des tests :

Utilisateur	Action	Résultat
lucas	clone	OUI
lucas	push	OUI
paul	clone	OUI
paul	push	NON

```
user@server:~/tp-dev-paul$ git push origin main
Username for 'http://localhost:3000': paul
Password for 'http://paul@localhost:3000':
Énumération des objets: 5, fait.
Décompte des objets: 100% (5/5), fait.
Compression des objets: 100% (3/3), fait.
Écriture des objets: 100% (3/3), 354 octets | 354.00 Kio/s, fait.
Total 3 (delta 0), réutilisés 0 (delta 0), réutilisés du paquet 0 (depuis 0)
remote:
remote: Forgejo: User permission denied for writing.
To http://localhost:3000/forgejo/tp-dev.git
 ! [remote rejected] main -> main (pre-receive hook declined)
erreur : impossible de pousser des références vers 'http://localhost:3000/forgejo/tp-dev.git'
```

Figure 20: Impossibilité pour paul de push

9.6.4 Problèmes rencontrés pendant les tests

9.6.4.1 Obligation de modifier le mot de passe avant une quelconque action

```
Clonage dans tp-dev-paul ...  
Username for 'http://localhost:3000': paul  
Password for 'http://paul@localhost:3000':  
remote: Réinitialisez votre mot de passe  
fatal : Échec d'authentification pour 'http://localhost:3000/forgejo/tp-dev.git/'
```

Figure 21: Problème mot de passe paul

Résolution

Simplement se connecter sur l'interface web avec le compte paul et changer le mot de passe comme demandé.

9.6.5 Création d'une issue

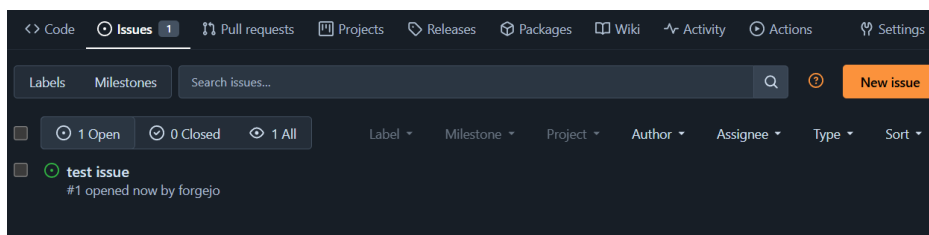


Figure 22: Issue créée

A l'aide de la commande suivante il est possible de résoudre l'issue. `> git commit -m "Fix #1 : ..."`

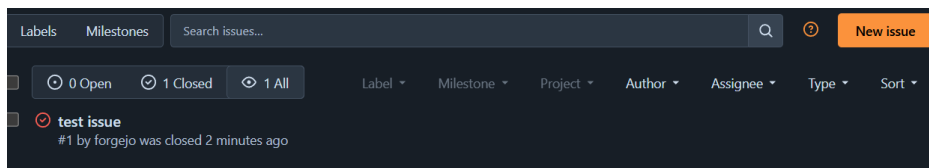


Figure 23: Issue fermée

9.6.6 Ajout d'une clé SSH

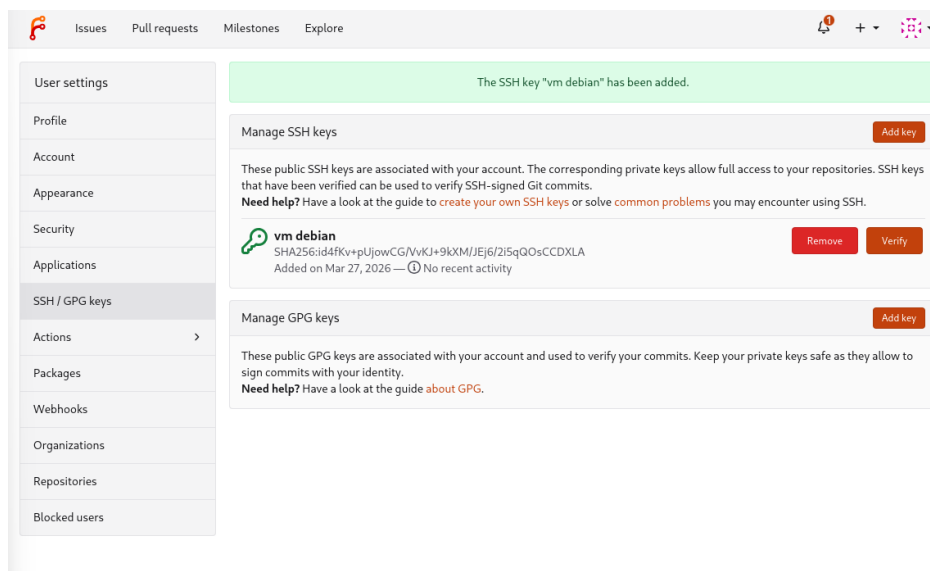


Figure 24: Ajout de la clé ssh

	Nom	Protocole	IP hôte	Port hôte	IP invité	Port invité
	Forgejo	TCP		3000		3000
	Forgejo ssh	TCP		2222		22

Figure 25: Ajout de la règle ssh dans VBox

Résultat : Le push et le clone s'effectuent correctement et sans mot de passe.

9.6.7 Questions

Question 1 : Créez un projet privé et un projet public. Quelle est la différence au niveau de l'accès ?

- Public : visible par tout le monde, même sans compte
- Privé : accessible uniquement aux membres explicitement autorisés

Vérification : en se déconnectant de Forgejo, le projet public reste accessible tandis que le projet privé renvoie une erreur 404.

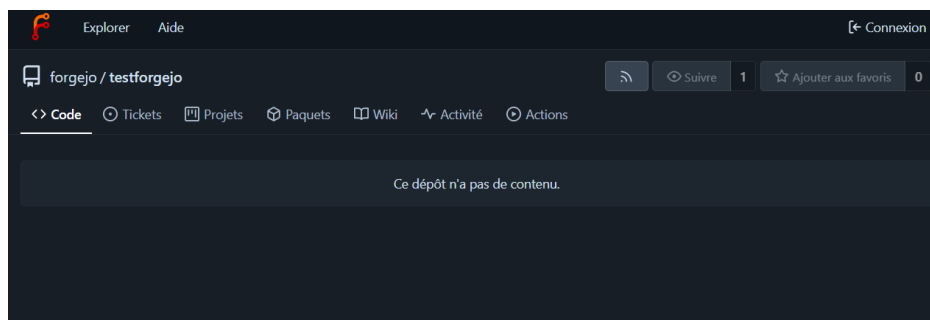


Figure 26: Projet en visibilité publique

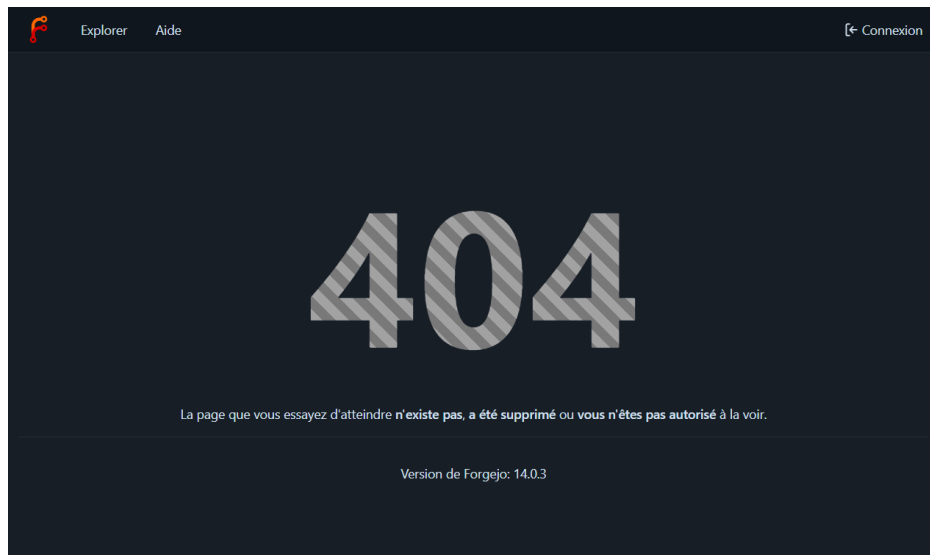


Figure 27: Projet en visibilité privée

10 Pour aller plus loin

10.1 Workflow Forgejo Actions

10.1.1 Création du workflow

Création du fichier `.forgejo/workflows/ci.yml` dans le dépôt :

```
mkdir -p .forgejo/workflows
```

Contenu du fichier `ci.yml` :

```
name: CI
on: [push]
jobs:
  build:
    runs-on: host
    steps:
      - name: Vérification
        run: ls -la
      - name: Test
        run: echo "Build réussi"
```

```
git add .forgejo/
git commit -m "Ajout du workflow CI"
git push forgejo main
```

Attention : Forgejo Actions nécessite qu'un runner soit configuré sur le serveur. Si aucun runner n'est disponible dans l'environnement de TP, le workflow reste en attente d'exécution. C'est une limitation de l'environnement, pas une erreur de configuration.

10.1.2 Création du runner

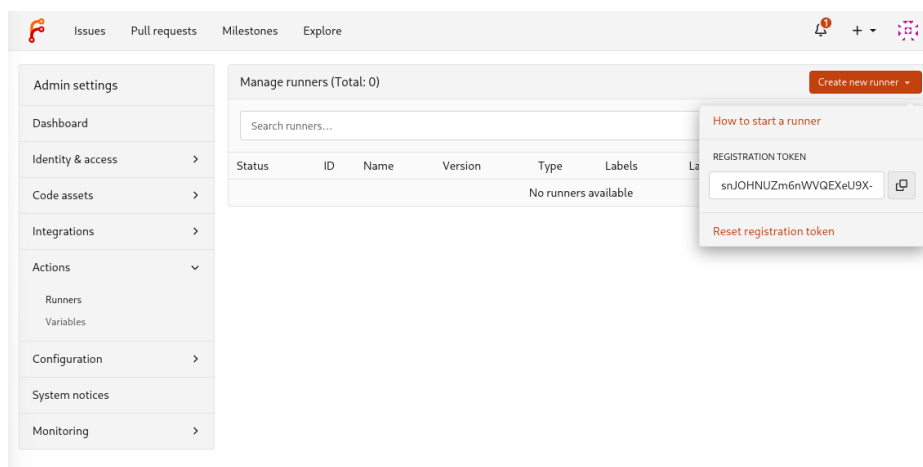


Figure 28: Récupération du token du runner

Installer le dernier binaire sur le site de Forgejo, le copier et le rendre executable :

```
cp forgejo-runner /usr/local/bin/forgejo-runner
sudo chmod 755 /usr/local/bin/forgejo-runner
```

Tester en tapant la commande :

```
forgejo-runner -v
```

Se rendre dans le terminal de l'utilisateur Git :

```
sudo -u git bash
cd /home/git
```

Configurer le runner :

```
forgejo-runner register --no-interactive --instance http://localhost:3000 \
--token LE_TOKEN --name runner-local --labels "host:host"
```

Lancer le runner :

```
forgejo-runner daemon
```

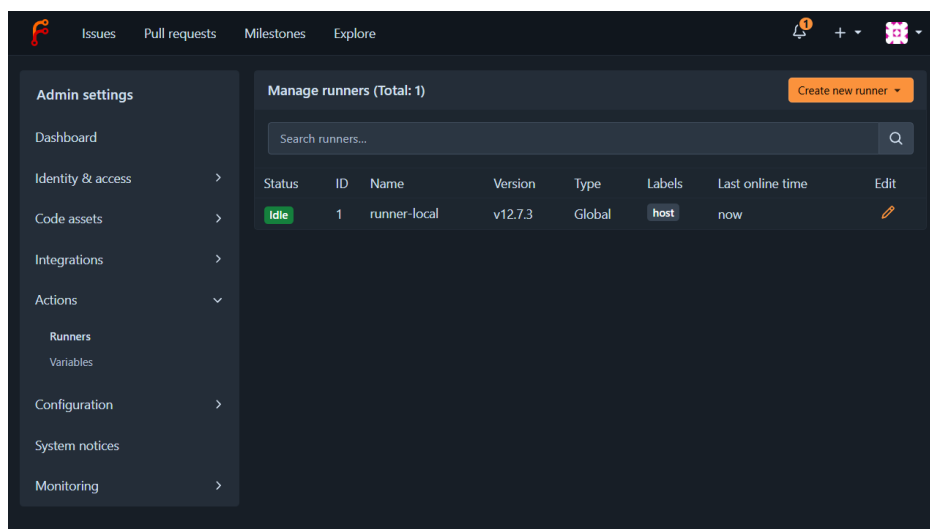


Figure 29: Création du runner

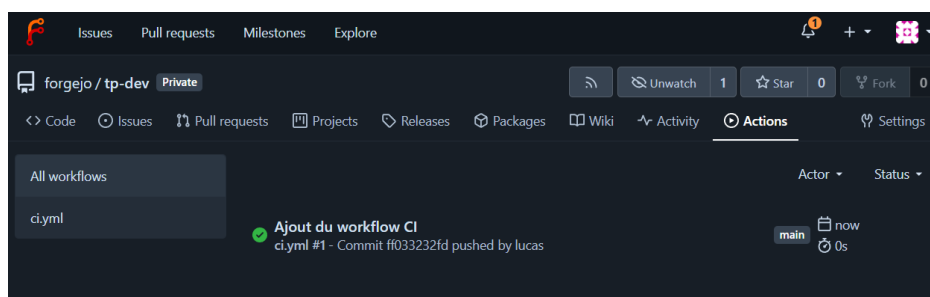


Figure 30: Workflow exécuté avec succès

10.2 Questions

Question 1 : Qu'est-ce que l'intégration continue (CI) ? La livraison continue (CD) ?

- CI (Intégration Continue) : à chaque push, des tests/vérifications automatiques sont lancés
- CD (Livraison Continue) : après validation des tests, le code est automatiquement prêt à être déployé

Question 2 : Quels sont les avantages de la CI/CD dans un projet en équipe ?

- Détection immédiate des erreurs après chaque push
- Garantie que la branche main est toujours fonctionnelle
- Personne n'oublie de lancer les tests car ils sont automatiques
- Historique complet de tous les builds

Source : Forgejo Docs - Forgejo Actions